



Today's agenda

↳ Shortest Path Algo

↳ Dijkstra's

↳ Bellman Ford



AlgoPrep



↳ Shortest Path Algo

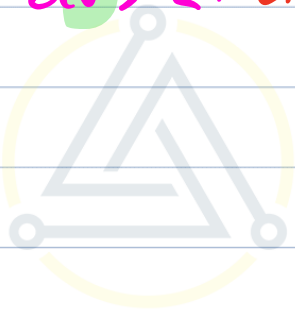
↳ BFS: Shortest Path in terms of edge count.

In terms of edge weights

$O(E \log V)$ ←↳ Dijkstra's: Single Src Shortest Path algo, only for non-negative edge weights.

$O(E \cdot V)$ ←↳ Bellman ford: Single Src Shortest Path algo, for -ve edge wts as well.
↳ $O(E \cdot V) \rightarrow O(E \cdot V^2)$
↳ $O(V^2 \cdot V) \rightarrow O(V^3)$

$O(V^3)$ ←↳ Floyd warshall: Any to Any Shortest Path algo, +ve, 0, -ve for all edge wts.

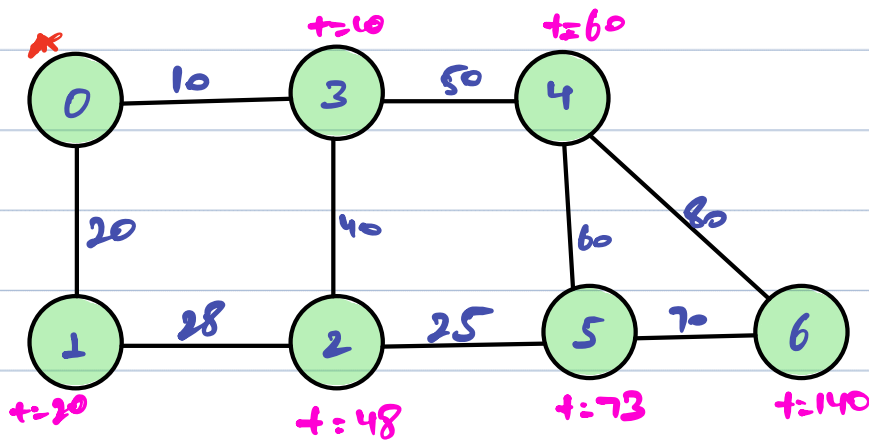


AlgoPrep



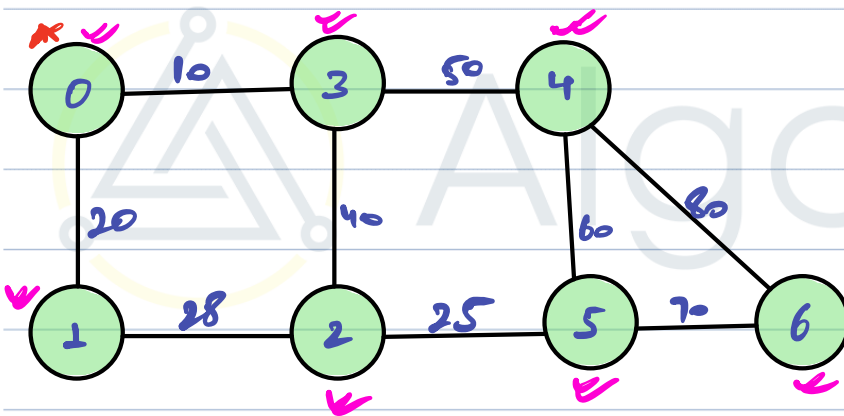
↳ Dijkstra's algo

→ $src = 0$

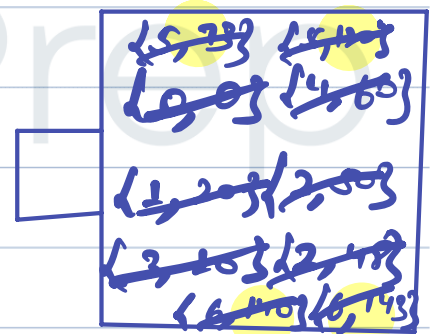


$t=10 \rightarrow 3$

$t=20 \rightarrow 1$



$\{v, wt\}$ $\leftarrow \min PQ$



0	1	2	3	4	5	6
0	20	48	10	60	73	140

rem: $\{5, 73\}$



// Pseudo code

```
class Pair1 {  $\rightarrow$  Graph  
    int v;  
    int w;  
}
```

```
class Pair2 {  $\rightarrow$  PQ  
    int v + w;  
    int w;  
}
```

```
int[] Dijkstra (List<List<Pair1>> graph, int src) {
```

```
    PriorityQueue<Pair2> PQ;
```

```
    int ans[n] = {+∞};
```

```
    PQ.add(new Pair2(src, 0));
```

$T.C: O(E \log V)$

$S.C: O(E)$

```
    while (PQ.size() > 0) {
```

```
        Pair2 rem = PQ.remove();  $\rightarrow$  rem.v + w  
                                 $\rightarrow$  rem.w
```

```
        if (ans[rem.v + w] != +∞) { continue; }
```

```
        ans[rem.v + w] = rem.w;
```

```
        List<Pair1> nbs = graph.get(rem.v + w);
```

```
        for (Pair1 n : nbs) {
```

```
            if (ans[n.v] != +∞) { continue; }
```

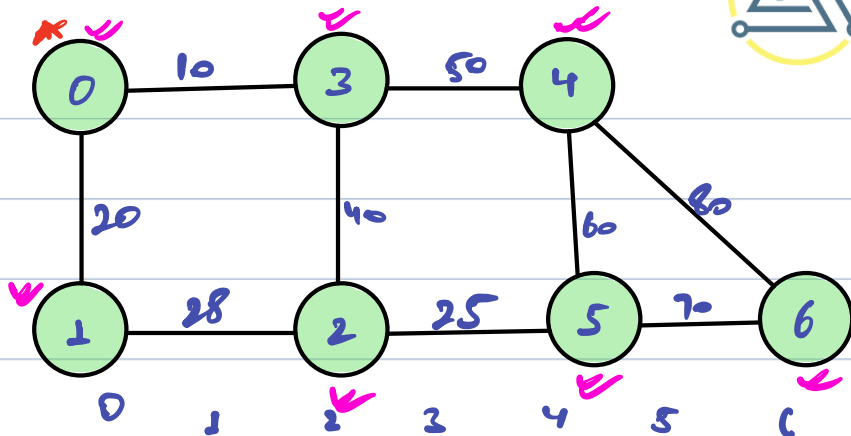
```
            PQ.add(new Pair2(n.v, rem.w + n.w));
```

```
    }
```

```
    }
```

```
    return ans;
```

Tracing



```

class Pair1 { → Graph
    int v;
    int w;
}

class Pair2 { → PQ
    int v;
    int w;
}
    
```

int[] Dijkstra (List<List<Pair1>> graph, int src)

PriorityQueue<Pair2> PQ;

int ans[n] = {∞};

PQ.add(new Pair2(src, 0));

while (PQ.size() > 0) {

Pair2 rem = PQ.remove();

if (ans[rem.v] != ∞) continue;

List<Pair1> nbs = graph.get(rem.v);

for (Pair1 n : nbs) {

if (ans[n.v] != ∞) continue;

PQ.add(new Pair2(n.v, rem.w + n.w));

return ans;

0	1	2	3	4	5	6
{0,0}	{0,10}	{1,28}	{2,40}	{3,50}	{4,60}	{5,70}
{1,20}	{2,28}	{3,40}	{4,50}	{5,60}		

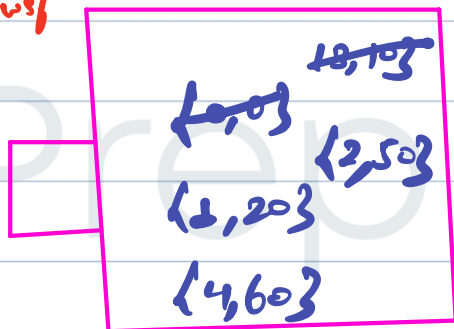
Pair1

rem = {3, 10}

rem.v

rem.w

min PQ(Pair2)



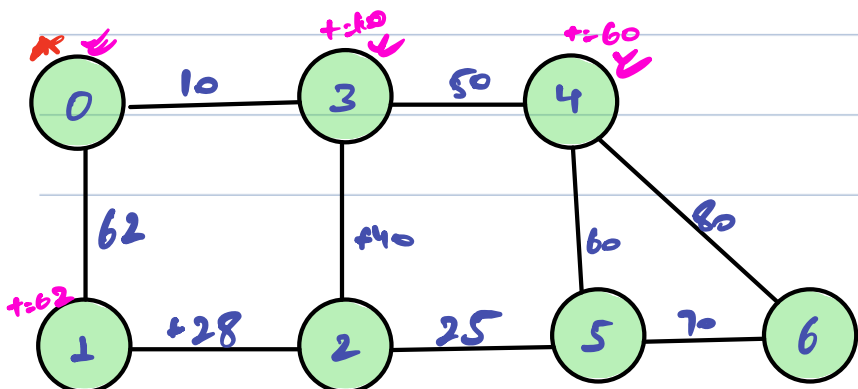
0	1	2	3	4	5	6
∞	∞	∞	10	∞	∞	∞

nbs → {2,40} {0,10} {4,50}

n.v must

TC: $O(E \log E) \rightarrow E = O(V^2)$

$E \log V^2 = 2E \log V = O(E \log V)$

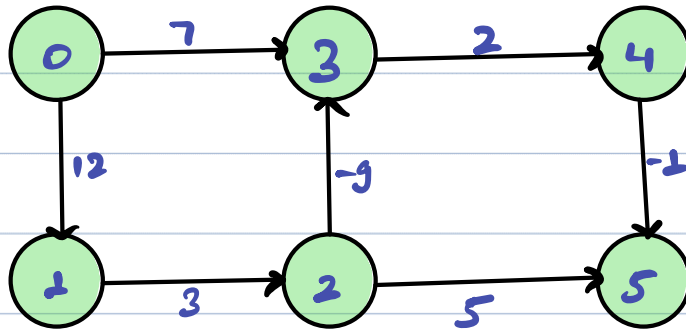


→ works only for the edge wt.



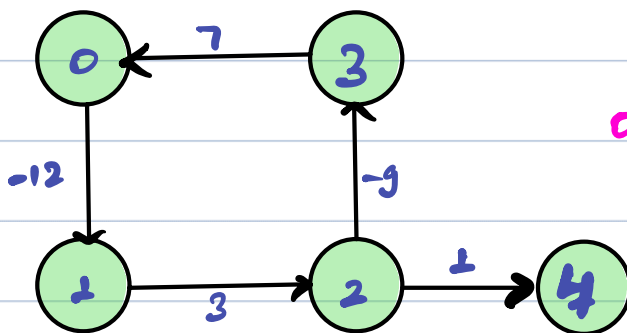
Q) Bellman Ford

↳ Shortest Path from a given src to all the remaining nodes in terms of edge weight.



obs 1:

Cycle in graph with overall -ve weight, Can we have shortest path?



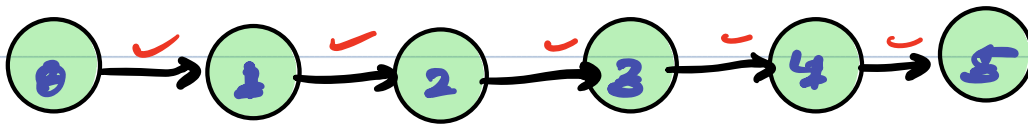
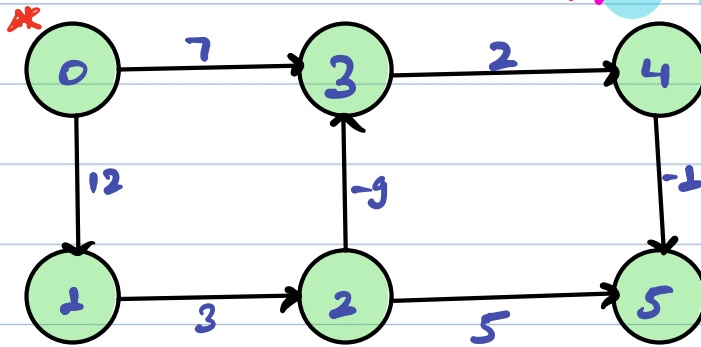
$0 \rightarrow 2 \rightarrow -9$

$0 \rightarrow 4 \rightarrow -8, -20, -32, \dots$

→ Shortest Paths are not possible for such case.



Obs 2: What can be the path length in a graph while finding shortest path.
↳ No. of nodes - 1.

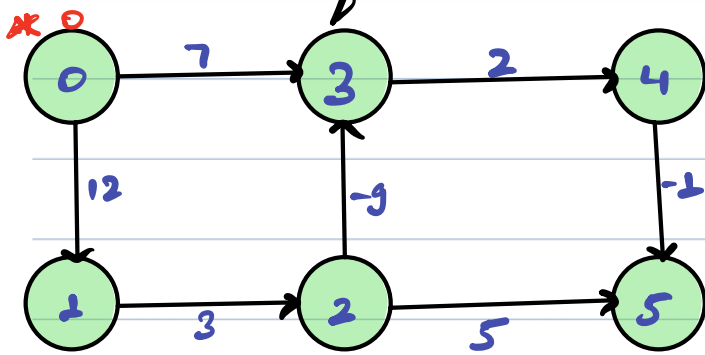


↳ No. of nodes - 1.

There is no cycle possible.



→ Bellman / Ford



u	v	w
4	5	-1
2	5	5
3	4	2
2	3	-9
1	2	3
0	1	12
0	3	7

ans:

0	1	2	3	4	5
0	12	15	7 6	∞ 9	∞ 8

→
 $\text{if}(\text{ans}[u] + w < \text{ans}[v])$
 $\text{ans}[v] = \text{ans}[u] + w;$

iter 1: 1 and 3

iter 2: 2 and 4

iter 3: 3 and 5

⋮

iter v-1:



11PSuedo Code

```
int[] Bellmanford (int [E][3] edges, int src, int v ) {
```

```
    int ans[v] = {∞};  
    ans[src] = 0;
```

```
    for (int i = 0; i < v-1; i++) { → iterations
```

```
        for (int j = 0; j < E; j++) {
```

```
            int u = edges[j][0];
```

```
            int v = edges[j][1];
```

```
            int wt = edges[j][2];
```

```
            if (ans[u] != ∞ && ans[u] + wt < ans[v]) {  
                ans[v] = ans[u] + wt;
```

```
            }
```

```
        }
```

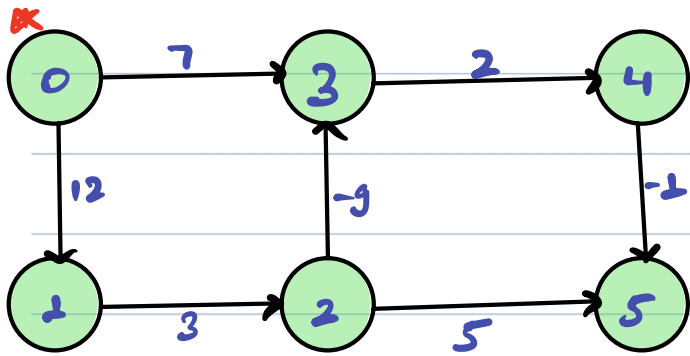
```
    }
```

```
    return ans;
```

```
}
```

T.C: $O(V \times E)$

S.C: $O(V)$



ans

0	1	2	3	4	5
0	12	15	7	8	20
					7

u	v	w*
4	5	-1
2	5	5
3	4	2
2	3	-9
1	2	3
0	1	12
0	3	7



→ It is Possible that you will get all your answer in 1st iteration, we do $v-1$ iterations considering the worst case scenario.



Q) Negative wt. cycle detection.

```
void negwtcycledetect (int [E][3] edges, int src, int v ) {
```

```
    int ans[v] = {∞};
```

```
    ans[src] = 0;
```

```
    for (int i=0; i<v-1; i++) { → iterations
```

```
        for (int j=0; j<E; j++) {
```

```
            int u = edges[j][0];
```

```
            int v = edges[j][1];
```

```
            int wt = edges[j][2];
```

```
            if (ans[u] != ∞ && ans[u] + wt < ans[v]) {
```

```
                ans[v] = ans[u] + wt;
```

```
            }
```

```
        }
```

```
    }
```

```
    boolean flag = false;
```

```
    for (int j=0; j<E; j++) {
```

```
        int u = edges[j][0];
```

```
        int v = edges[j][1];
```

```
        int wt = edges[j][2];
```

```
        if (ans[u] != ∞ && ans[u] + wt < ans[v]) {
```

```
            flag = true; break;
```

```
        }
```

```
    }
```

```
    if (flag == true) { s.o.p("negative wt. cycle"); }
```

```
}
```

T.C: $O(V \times E)$

S.C: $O(V) + 1$